

École Hassania des Travaux Publics (EHTP)

Casablanca, Morocco

Engineering Cycle — Computer Engineering

Specification Document

End-to-End DevSecOps Platform

Securing the Software Supply Chain

Prepared by: Amhidi Hamza

Domain: Cybersecurity — DevSecOps — AppSec

Version: 1.0

Date: September 2026

Academic year: 2026–2027

Code → Build → Security analysis → Container security → IaC security → DAST → Deployment →
Monitoring

Contents

1	Introduction	3
1.1	Context	3
1.2	Problem Statement	3
1.3	Objectives	3
1.4	Scope	3
2	Project Overview	4
3	Requirements Analysis	4
3.1	Actors	4
3.2	Functional Requirements	4
3.3	Non-Functional Requirements	5
4	Technical Architecture	5
4.1	Pipeline Stages	6
4.2	Environments	6
5	Detailed Description of the Modules	6
5.1	Target Application: SecureBank	6
5.2	Red / Blue Scenario	7
5.3	Pipeline Security Modules	7
5.3.1	SAST — Static Analysis	7
5.3.2	SCA — Dependency Security	7
5.3.3	Secret Detection	7
5.3.4	Container Security	8
5.3.5	SBOM	8
5.3.6	Infrastructure as Code Security	8
5.3.7	DAST — Dynamic Analysis	8
5.4	Risk Engine	8
5.5	Detection and Monitoring	9
6	Technology Stack	9
7	Dependencies	10
7.1	Dependencies Between Modules	10
7.2	External Dependencies and Prerequisites	11
8	Planning	11
9	Deliverables	11
10	Validation Criteria and Indicators	12
11	Risks and Mitigation Measures	12
12	Ethical and Legal Constraints	12
13	Conclusion	12
A	Appendix A — Indicative GitLab CI Pipeline Skeleton	13

B Appendix B — Risk Engine Pseudo-code**13**

1 Introduction

1.1 Context

Modern applications are delivered faster and faster through continuous integration and continuous deployment (CI/CD) pipelines. Every stage of this chain (source code, dependencies, container image, infrastructure, runtime environment) is a potential attack surface. The *DevSecOps* approach consists in integrating security from the very beginning of the software lifecycle (the *shift-left* approach) rather than treating it at the end of the project.

This project aims to design a platform that illustrates both the **offensive side** (identifying and exploiting vulnerabilities) and the **defensive side** (secure software delivery, detection and monitoring).

1.2 Problem Statement

Central question

How can an application's security analysis be automated throughout its delivery, and above all how can risks be **prioritized** and a deployment **blocked** on the basis of real, justifiable risk rather than a simple severity score?

1.3 Objectives

General objective. Build a complete DevSecOps platform that, on every code change, automatically runs a series of security checks and decides whether deployment is allowed.

Specific objectives.

- Provide an intentionally vulnerable target application (*SecureBank*) that makes it possible to demonstrate real attacks.
- Integrate SAST, SCA, secret detection, image scanning, SBOM generation, IaC security and DAST into a GitLab CI pipeline.
- Develop a Python **risk engine** that aggregates the results, computes a contextualized risk score and decides whether to block or allow deployment.
- Deploy the application on Kubernetes and detect attack behavior through log analysis (brute force, IDOR attempts).
- Demonstrate the complete scenario: attack, finding, fix, commit, pipeline validation, deployment.

1.4 Scope

In scope	Vulnerable application, secured CI/CD pipeline, risk engine, Kubernetes deployment, log-based detection, documentation and demonstration.
Optional (if time permits)	Runtime security with Falco, full Terraform infrastructure, full ELK/OpenSearch stack, Prometheus/Grafana dashboards.
Out of scope	Penetration testing of third-party systems, paid proprietary security tools, formal regulatory compliance.

2 Project Overview

A developer pushes code to the GitLab repository. The pipeline is triggered automatically and takes the code through the following stages: **Code** → **Build** → **Security analysis** → **Container security** → **IaC security** → **DAST** → **Deployment** → **Monitoring**.

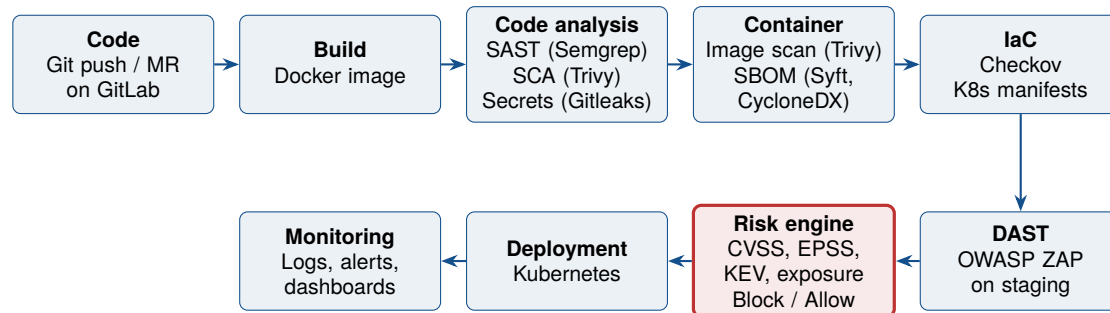


Figure 1: Platform overview (pipeline flow)

Guiding principle

The project does not simply run tools one after the other. Its value rests on an **intelligent** pipeline that applies an explicit security policy: the results of each tool are consolidated and then evaluated by a risk engine before any production release.

3 Requirements Analysis

3.1 Actors

- **Developer**: pushes code and fixes reported vulnerabilities.
- **Security engineer (blue team)**: defines the security policy, tunes thresholds and handles alerts.
- **Pentester (red team)**: attacks the vulnerable application and documents the findings.
- **Evaluator / supervisor**: reviews the documentation, the report and the demonstration.

3.2 Functional Requirements

Priorities: **M** = must have, **S** = should have, **C** = could have (optional).

Ref.	Description	Prio.
FR1	Provide the <i>SecureBank</i> target application with documented intentional vulnerabilities.	M
FR2	Automatically trigger the pipeline on every <i>push</i> or <i>merge request</i> .	M
FR3	Static analysis of the source code (SAST).	M
FR4	Analysis of vulnerable dependencies (SCA).	M
FR5	Detection of secrets in the repository; pipeline fails if a secret is committed.	M
FR6	Build the Docker image and scan it for vulnerabilities.	M
FR7	Generate an SBOM in CycloneDX format and correlate it with known vulnerabilities.	S
FR8	Security analysis of Kubernetes manifests (and Terraform as an option).	M
FR9	Dynamic testing (DAST) of the application deployed in staging.	M
FR10	Risk engine: report aggregation, score computation, block/allow decision.	M
FR11	Consolidated report of findings (readable and actionable format).	S
FR12	Conditional deployment on Kubernetes based on the risk engine's decision.	M
FR13	Rule-based attack detection on logs (brute force, IDOR).	S
FR14	Monitoring and dashboards (Prometheus, Grafana).	C
FR15	Runtime security (Falco).	C

Table 1: Functional requirements

3.3 Non-Functional Requirements

- **Reproducibility:** the whole platform must be re-runnable from the repository (containers, manifests, versioned configuration).
- **Traceability:** every blocking decision must be explainable (which findings, which parameters, which score).
- **Modularity:** each tool can be replaced without rewriting the risk engine (normalized input formats).
- **Performance:** reasonable pipeline duration for repeated use (indicative target to be measured and documented).
- **Quality of results:** limit false positives and alert noise.
- **Laboratory security:** the vulnerable application remains isolated and is never exposed to the Internet.
- **Documentation:** public repository with a clear README, architecture diagram and demonstration.

4 Technical Architecture

4.1 Pipeline Stages

Stage	Tool	Analysis	Blocking condition
Secrets	Gitleaks	Search for keys, passwords and tokens in the code and history	Any detected secret
SAST	Semgrep	SQL injection, XSS, command injection, weak cryptography, dangerous functions, authentication issues	Risk engine score
SCA	Trivy	Dependencies with known CVEs	Risk engine score
Image	Trivy	Vulnerabilities in the base system and image packages	Exploitable critical vulnerability
SBOM	Syft, CycloneDX	Component inventory and CVE correlation	Information for the risk engine
IaC	Checkov	Kubernetes/Terraform misconfigurations	Unjustified risky configuration
DAST	OWASP ZAP	SQL injection, XSS, authentication, security headers, exposed endpoints, API vulnerabilities	Risk engine score
Decision	Risk engine (Python)	Aggregation and scoring	Score above threshold

Table 2: Details of the security stages

4.2 Environments

- **Development:** local machine with Docker.
- **Continuous integration:** GitLab CI with a Docker *runner*.
- **Pre-production (staging):** ephemeral deployment on Kubernetes (for example kind or minikube) to run DAST.
- **Simulated production:** demonstration Kubernetes cluster, with monitoring.

5 Detailed Description of the Modules

5.1 Target Application: SecureBank

A simplified banking application, designed to be attacked.

Features: authentication, user management, REST API, money transfers, admin panel, PostgreSQL database, Docker containerization, Kubernetes deployment.

Intentional vulnerabilities and the tool responsible for detecting them:

Vulnerability	Planned detection	Comment
SQL injection	SAST (Semgrep) and DAST (ZAP)	Double detection possible
XSS	SAST and DAST	Double detection possible
Command injection	SAST	Dangerous function identifiable in the code
SSRF	SAST (targeted rules)	Basic DAST detects it poorly
JWT weaknesses	SAST (targeted rules)	Weak algorithm, hard-coded secret
IDOR	Automated API tests and log rules	Generic tools detect IDOR poorly
Broken access control	Automated API tests	Requires a business-specific test

Table 3: Vulnerabilities injected into SecureBank

Scope decision

For a realistic first milestone, the minimum goal is to include 4 to 5 real vulnerabilities (SQL injection, IDOR, JWT, SSRF, and XSS or command injection) before extending to the full list. Some logic vulnerabilities (IDOR, access control) are not found by a generic scanner: the project documents this honestly and covers them with dedicated tests.

5.2 Red / Blue Scenario

The demonstration scenario links the attack, the fix and the automatic validation:

Attacker → Vulnerable application → Penetration test → Finding → Developer fix → Git commit → Security pipeline → Security tests → Deployment

This scenario connects the **Red Team**, the **Blue Team** and **DevSecOps**. It will be documented as a detailed *before/after* for at least one vulnerability (exploitation, fix, blocking, then pipeline validation).

5.3 Pipeline Security Modules

5.3.1 SAST — Static Analysis

Analysis of the source code to detect: SQL injection, XSS, command injection, insecure cryptography, dangerous functions and authentication issues. Selected tool: **Semgrep** (alternatives considered: Bandit, SonarQube).

5.3.2 SCA — Dependency Security

Automatic analysis of the libraries in use. Example of the expected reasoning:

Flask (vulnerable version) → CVE → CVSS 9.8 → PIPELINE BLOCKED

Selected tool: **Trivy** (alternative: OWASP Dependency-Check).

5.3.3 Secret Detection

The pipeline must detect elements such as:

```
AWS_SECRET_KEY = "..."  
DATABASE_PASSWORD = "..."  
JWT_SECRET = "..."
```

Selected tool: **Gitleaks**. The pipeline fails as soon as a secret is committed.

5.3.4 Container Security

Sequence: `docker build` → Trivy scan → critical vulnerability? If so, block; otherwise, continue the pipeline.

5.3.5 SBOM

Generation of a software inventory (*Software Bill of Materials*) of the application's components (Flask, PostgreSQL, Requests, OpenSSL, Node.js, etc.) in **CycloneDX** format with **Syft**, followed by correlation with known vulnerabilities. This stage gives the project an enterprise-level security engineering dimension.

5.3.6 Infrastructure as Code Security

Creation of a Kubernetes infrastructure (and Terraform as an option) that deliberately contains misconfigurations, for example:

```
privileged: true
hostNetwork: true
runAsUser: 0          # or runAsNonRoot: false
```

These errors are automatically detected by **Checkov** and/or **Kubescape**.

5.3.7 DAST — Dynamic Analysis

After deployment in staging, **OWASP ZAP** explores the attack surface and tests: SQL injection, XSS, authentication, security headers, exposed endpoints and API vulnerabilities. Burp Suite is still used manually during penetration testing. This stage showcases penetration testing experience.

5.4 Risk Engine

A module developed in Python, at the heart of the project. Instead of simply reporting “critical vulnerability found”, it computes a contextualized risk score.

Initial formula. For a finding f , with all components normalized between 0 and 1:

$$R(f) = 10 \times (0.35c + 0.20e + 0.15k + 0.15x + 0.15a)$$

- c : CVSS score divided by 10 (technical severity);
- e : EPSS exploitation probability (0 to 1);
- k : 1 if the CVE appears in the CISA KEV catalog (confirmed exploitation), 0 otherwise;
- x : exposure (1 if exposed to the Internet, reduced value if internal);
- a : asset criticality (for example 1 for a production API, lower for a secondary component).

Decision policy.

- $R \geq 8$: **deployment blocked**;
- $6 \leq R < 8$: warning, human review required;
- $R < 6$: allowed.
- Overriding rule: a vulnerability present in the KEV catalog on an exposed asset triggers a block regardless of the score.

Calculation examples.

Finding	<i>c</i>	<i>e</i>	<i>k</i>	<i>x</i>	<i>a</i>	<i>R</i>	Decision
Critical CVE, confirmed exploit, exposed production API	0.98	0.70	1	1	1	9.3	Blocked
High CVE, low probability, internal component	0.75	0.02	0	0	0.5	3.4	Allowed

Table 4: Two scoring examples

Justification and limitations

The weights above are starting assumptions: they must be **calibrated and justified** on the set of vulnerabilities injected into SecureBank. The EPSS and KEV components only apply to findings tied to a CVE (SCA, image). For SAST, DAST and IaC findings, the base severity is derived from the tool's severity, and the EPSS/KEV components are neutralized with renormalization of the weights. Each decision is written to a report stating the values used.

How it works. The engine reads the normalized reports (Semgrep, Trivy, Gitleaks, Checkov, ZAP, SBOM), enriches the CVEs with EPSS and KEV, applies the formula, produces a consolidated report and returns an exit code that blocks or allows the deployment stage.

5.5 Detection and Monitoring

After deployment, the application generates security logs:

Application → Fluent Bit → ELK / OpenSearch → Detection rules → Dashboard

Planned detection rules:

- many failed logins in a short period (for example 100) → possible brute force → alert;
- multiple attempts to access other users' resources → suspicious IDOR-type activity → alert.

If the full ELK stack slows the project down, a minimal version (log rules + a small dashboard) is accepted. Prometheus and Grafana provide monitoring; Falco adds runtime detection (optional).

6 Technology Stack

The project does not need to use every tool: the goal is to demonstrate the concepts properly, with one main tool per category.

Area	Technology	Status
Backend	Python / FastAPI	Selected
Frontend	React	Selected
Database	PostgreSQL	Selected
Version control	GitLab	Selected
CI/CD	GitLab CI	Selected
Containers	Docker	Selected
Orchestration	Kubernetes	Selected
SAST	Semgrep	Selected (alternatives: Bandit, SonarQube)
SCA	Trivy	Selected (alternative: OWASP Dependency-Check)
Secrets	Gitleaks	Selected
DAST	OWASP ZAP	Selected (Burp Suite for manual testing)
IaC security	Checkov	Selected (alternative: Kubescape)
SBOM	Syft + CycloneDX	Selected
Container security	Trivy	Selected
Automation	Python (risk engine, scripts)	Selected
Infrastructure as code	Terraform	Optional
Runtime security	Falco	Optional
Monitoring	Prometheus + Grafana	Optional
Logs	Fluent Bit + OpenSearch / ELK	Optional (minimal version accepted)

Table 5: Project technologies

7 Dependencies

7.1 Dependencies Between Modules

Module	Depends on
CI/CD pipeline	GitLab repository, Docker <i>runner</i> , SecureBank application
Image scan and SBOM	Docker image built by the pipeline
IaC security	Kubernetes manifests (and Terraform) versioned in the repository
DAST	Application deployed in staging on Kubernetes
Risk engine	Normalized reports from all analysis tools, enrichment sources (NVD/CVSS, EPSS, CISA KEV), exposure and criticality metadata
Deployment	Risk engine decision, Kubernetes cluster
Detection and monitoring	Structured application logs, collection (Fluent Bit), storage/search (OpenSearch or equivalent)

Table 6: Internal dependencies

7.2 External Dependencies and Prerequisites

- **Vulnerability databases:** NVD/CVSS, EPSS (FIRST) and the CISA KEV catalog, with network access or a local copy to limit network dependency.
- **Infrastructure:** Docker, a local Kubernetes cluster (kind or minikube), a GitLab instance with a *runner*.
- **Security tools:** Semgrep, Trivy, Gitleaks, Syft, Checkov, OWASP ZAP (versions pinned in the repository for reproducibility).
- **Languages and libraries:** Python (FastAPI, risk engine), JavaScript/React (user interface).
- **Skills:** web penetration testing, Docker/Kubernetes, CI/CD, Python.

8 Planning

Estimated duration: 12 to 14 weeks.

Week	Tasks	Deliverable
W1-2	Scoping, GitLab repository, SecureBank application with 4 to 5 vulnerabilities, containerization	Dockerized application
W3-4	Basic pipeline: Gitleaks, Semgrep, Trivy (dependencies)	SAST/SCA/secrets pipeline
W5-6	Image scan, SBOM, IaC security (Checkov), Kubernetes manifests	Container and IaC pipeline
W7-8	DAST (ZAP), risk engine development (CVSS, EPSS, KEV, exposure)	Working risk engine
W9-10	Conditional Kubernetes deployment, log detection rules	End-to-end platform
W11-12	Testing, measurements, before/after scenario, documentation, demo video	Public repository and report

Table 7: Provisional schedule

Extensions if time permits: Falco, full Terraform, full ELK, Prometheus/Grafana.

9 Deliverables

- Source code of the SecureBank application and its fixes.
- GitLab CI pipeline file and associated scripts.
- Python risk engine code, with its documentation and test suite.
- Kubernetes manifests (and Terraform as an option).
- Detection rules and dashboard.
- Public repository with a clear README and architecture diagram.
- Short demonstration video or GIF.
- Written report including the before/after study of one vulnerability (exploitation, fix, blocking, validation).

10 Validation Criteria and Indicators

The project's success will be measured through concrete results:

- detection rate on the injected vulnerabilities (for example x out of 12 detected);
- number of false positives and noise reduction achieved thanks to the risk engine compared with blocking on severity alone;
- pipeline execution time;
- effective blocking of a deployment containing a critical vulnerability, then authorization after the fix;
- detection alerts triggered during simulated attacks;
- reproducibility: a third party can re-run the platform by following the README.

11 Risks and Mitigation Measures

Risk	Mitigation measure
Scope too broad (more than 15 integrations)	One tool per category, optional modules postponed, go deeper rather than wider
Unjustified risk engine weights	Calibration on a known set of vulnerabilities, reliance on recognized sources (CVSS, EPSS, KEV)
Many false positives	Tuned rules, documented exception list, noise measurement
Unavailability of EPSS/KEV/NVD sources	Local copy of the data, documented degraded mode
Complexity of the logging stack	Minimal version of the detection rules
Pipeline too slow	Parallel analyses, caching, analyses targeted at the changes
Overlap with existing projects	Emphasize the risk engine and detection, not a generic pipeline

Table 8: Risk analysis

12 Ethical and Legal Constraints

- Attacks target only the SecureBank application, hosted in an isolated environment.
- No scan or penetration test is carried out on third-party systems without authorization.
- The vulnerable application is never publicly exposed and the demonstration secrets are fictitious.
- The tools used are open source or have a license suited to academic use.

13 Conclusion

This project brings together penetration testing, application security, DevSecOps, cloud security and detection within a single platform. Its value rests less on the number of tools than on three elements: a complete attack–fix–validation scenario, a risk engine justified by real data, and a detection layer. This specification document defines the scope, architecture, dependencies and schedule required to carry it out.

A Appendix A — Indicative GitLab CI Pipeline Skeleton

Schematic skeleton to be adapted (job names and options to be validated during implementation):

```
stages:
  - secrets
  - sast
  - sca
  - build
  - image-scan
  - sbom
  - iac
  - deploy-staging
  - dast
  - risk-gate
  - deploy

secrets:
  stage: secrets
  script: gitleaks detect --source . --report-path gitleaks.json

sast:
  stage: sast
  script: semgrep --config auto --json -o semgrep.json .

sca:
  stage: sca
  script: trivy fs --format json -o trivy-fs.json .

image-scan:
  stage: image-scan
  script: trivy image --format json -o trivy-image.json $IMAGE

sbom:
  stage: sbom
  script: syft $IMAGE -o cyclonedx-json > sbom.json

iac:
  stage: iac
  script: checkov -d k8s/ -o json > checkov.json

dast:
  stage: dast
  script: zap-baseline.py -t $STAGING_URL -J zap.json

risk-gate:
  stage: risk-gate
  script: python risk_engine.py --reports reports/ --threshold 8
```

B Appendix B — Risk Engine Pseudo-code

```
WEIGHTS = {"cvss": 0.35, "epss": 0.20, "kev": 0.15,
           "exposure": 0.15, "criticality": 0.15}

def risk_score(f):
    return 10 * (WEIGHTS["cvss"] * f.cvss / 10
                + WEIGHTS["epss"] * f.epss
                + WEIGHTS["kev"] * f.kev
                + WEIGHTS["exposure"] * f.exposure
                + WEIGHTS["criticality"] * f.criticality)
```

```
def decide(findings):  
    scores = [risk_score(f) for f in findings]  
    if any(f.kev and f.exposure for f in findings) or max(scores, default=0) >= 8:  
        return "BLOCK"  
    if max(scores, default=0) >= 6:  
        return "WARN"  
    return "PASS"
```